

Lecture 2: Search

Helen Harman

CMP2020 Artificial Intelligence

541625

Classify the environment that a thermometer operates in.

Whilst you wait for the lecture to start, enter you answers:



pollEv.com/HelenHarman

Hint:

Fully observable or partially observable

Deterministic or non-deterministic

Episodic or sequential

Static or dynamic

Discrete or continuous

Thermometer environment classification

This is one answer – you could argue for other classifications

Fully or partially observable: fully observable (only needs to measure temperature)

Deterministic or non-deterministic: deterministic (if it is calibrated correctly)

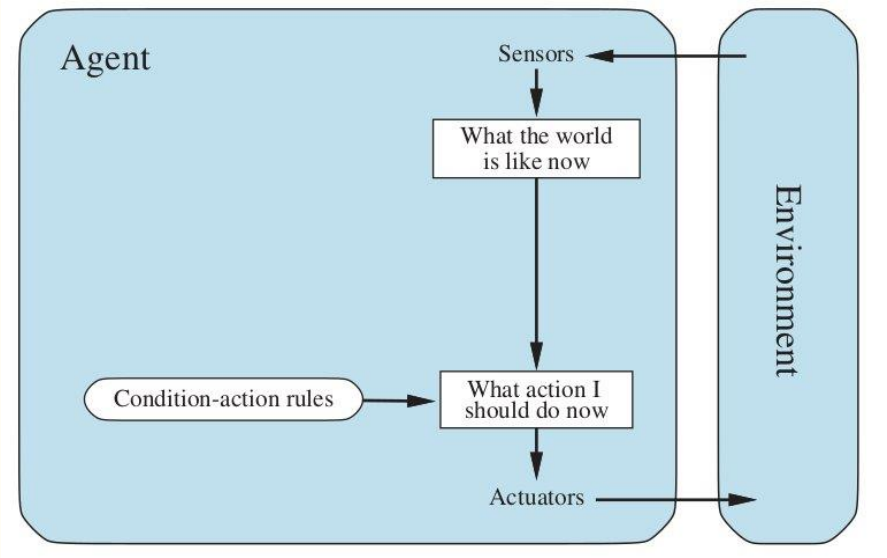
Episodic or sequential: Episodic (each temperature reading is independen

Static or dynamic: Dynamic (temperature can change whilst it is bein

Discrete or continuous: Continuous (temperature is a continuous variable)

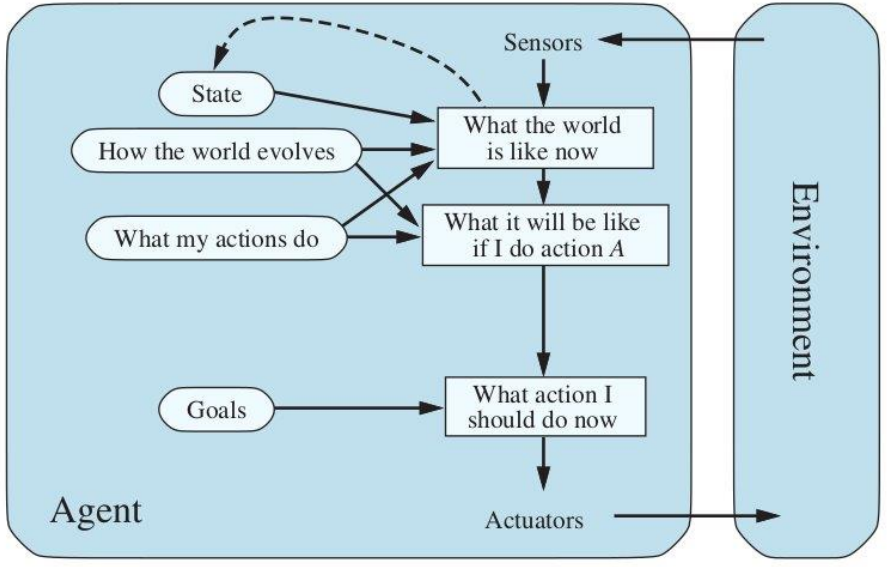
Last week

Reflex agent



This week

Problem solving agent



Lecture Learning Outcomes

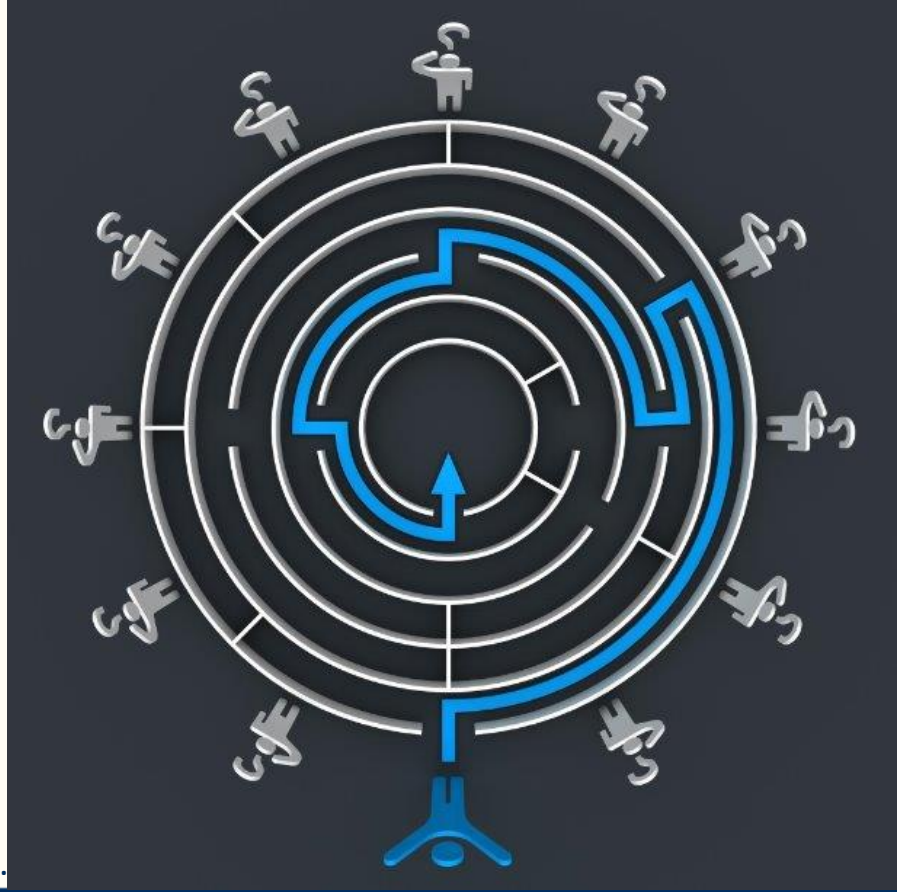
By the end of this lecture, you will be able to:

- Understand search as a problem-solving technique in AI.
- Represent problems as graphs.
- Explain DFS, BFS, DLS and IDS.
- Compare the strengths and weaknesses of each method.

What is Search in AI?

- Problem-solving agent: Takes input (goal, environment) → outputs a sequence of actions.
- Explores possible solutions to find a path from initial to goal states
- Solve navigation, puzzles, routing and task planning (e.g. in robotics, logistics, gaming) problems in real-world AI systems.

Search methods takes a *problem* as input and search for a solution.



(copilot, 2025)

Problem Formulation in AI

Initial State Definition

The initial state represents the starting point of the AI problem.

Actions and Operators

Actions define possible moves or operations to transition between states in the problem space.

Transition Model

The transition model describes the outcome of applying actions, mapping states to new states.

Goal Test and Path Cost

Goal test checks if the objective is met. Path cost evaluates numeric cost for solution paths.

(copilot, 2025)

An example problem

Imagine we are in Grimsby.

We have reasoned about the different activities that we could do today.

We have decided our highest priority is to get to Nottingham (e.g. because we have a flight booked).



Stock beach image... not of Grimsby

Problem Formulation: Initial state

Our initial state:

In(Grimsby)

Could include more details:

Temperature(low)
Weather(wet)
Shops(open)

Finding the correct level of abstraction is important.

Problem Formulation: Actions

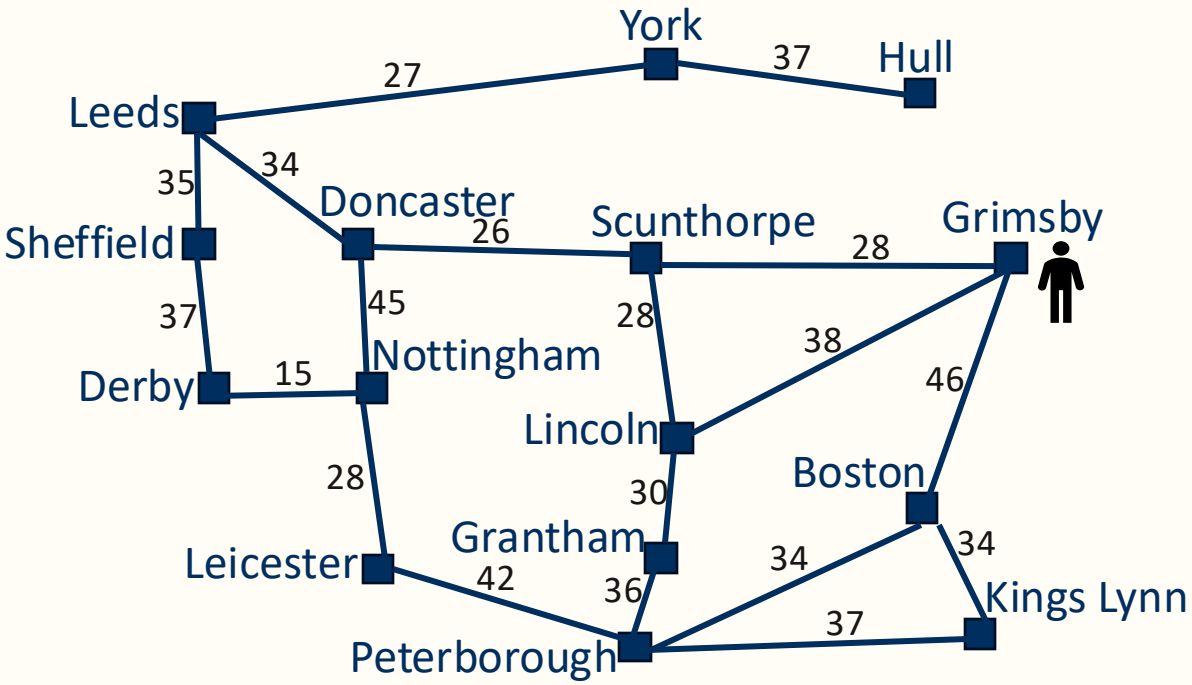
Dependent on what state the agent is in.
Given a particular state `s`, `Actions(s)` is the set of actions that are applicable.

In our example:

{Go(Scunthorpe), Go(Lincoln), Go(Boston)}

Again, level of abstraction is important.

Problem Formulation: Example



Problem Formulation: Transition model

Describes what each action does.

- Formally: `Result(s, a)`
- This defines the state the agent reaches by executing action `a` in state `s`.
- It produces a *successor state*.
- Deterministic environments have a single successor.

Result(In(Grimsby), Go(Boston)) = In(Boston)

Problem Formulation: Goal test

Has the goal been reached?

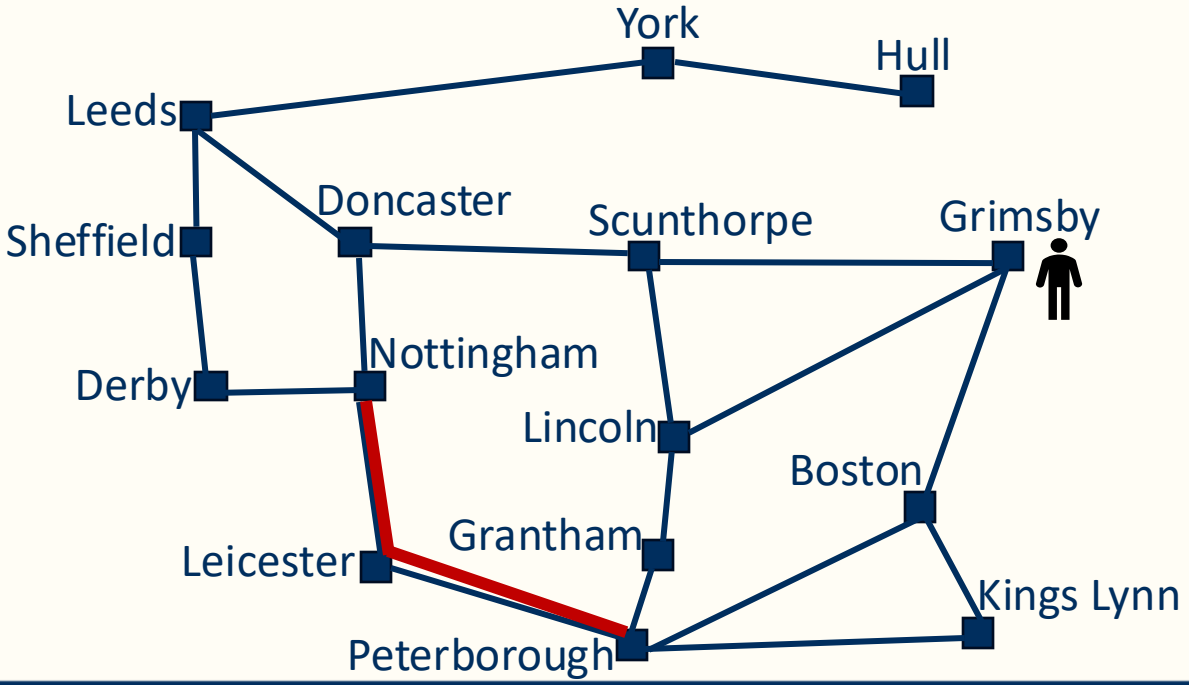
Our goal: *{In(Nottingham)}*

Goal test: *Equal(state, {In(Nottingham)})*

In other domains, could use for example: "contains", "does not contain" etc.
If state contained "In(NottinghamCastle)", the agent would need to use inference/rules to determine that Nottingham Castle is in Nottingham.

Problem Formulation: Path cost

A path in a state space is a sequence of actions and states



The step cost of taking action `a` in state `s` to reach state `s'` is written as: `c(s,a,s')`

Path cost: Sum of all step costs in the path.

path cost = c(in(P), go(L), in(L))
+ c(in(L), go(N), in(N))

Could use the distance travelled.

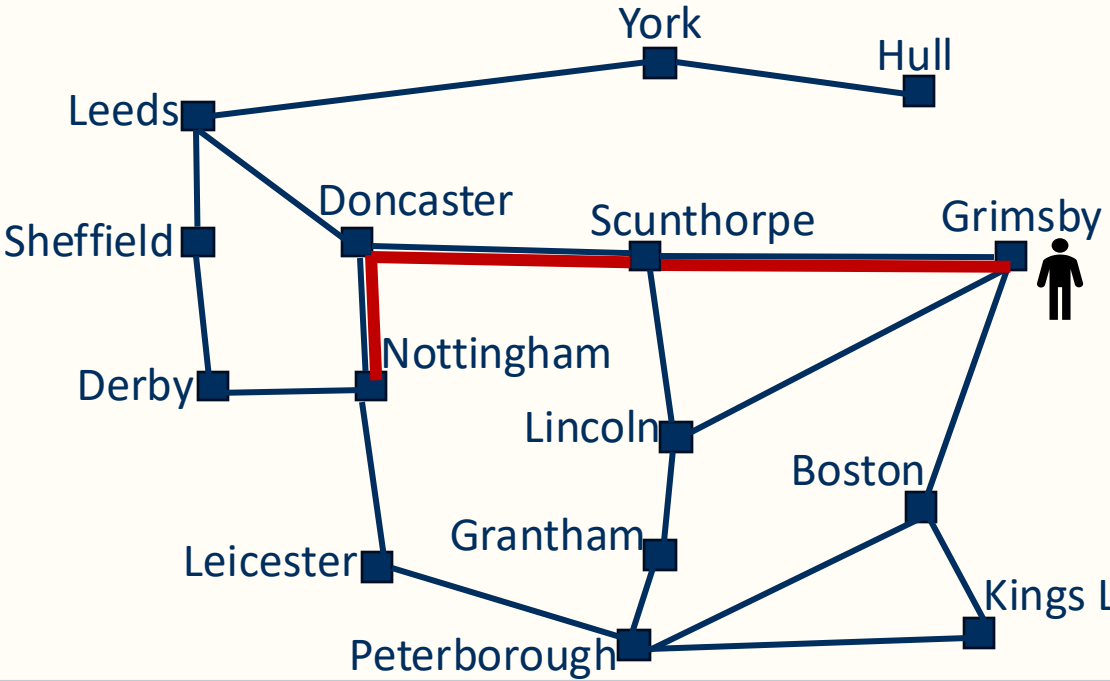
Peterborough shortened to P, Leicester to L and Nottingham to N.

Problem Formulation

- A solution is a path (sequence of actions) from the initial state to the goal -- a plan.
- Quality: path cost
- Optimal plan: plan with the lowest path cost

Plan

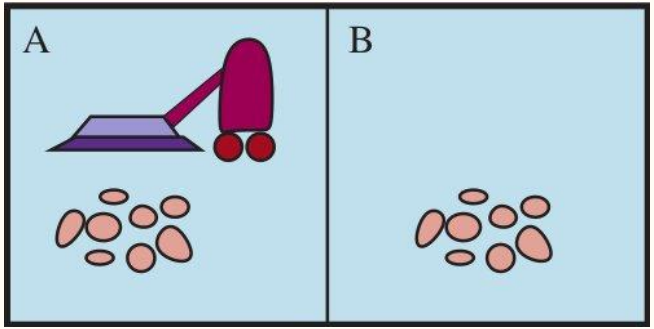
A path from the initial state to the goal is a plan.



In(Grimsby)
Go(Scunthorpe)
In(Scunthorpe)
Go(Doncaster)
In(Doncaster)
Go(Nottingham)
In(Nottingham)

Plan cost= 3

Example: Vacuum World



What would the Problem Formulation of vacuum world look like?

Problem Formulation: Vacuum World

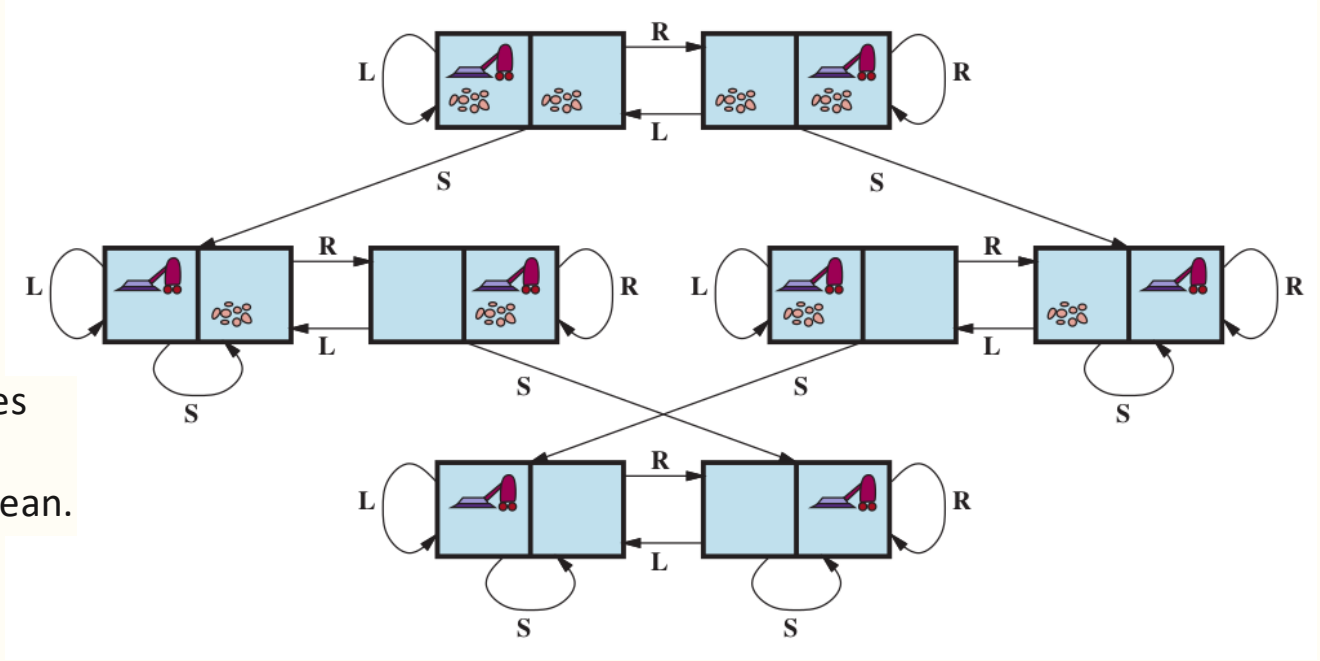
States: 8 possible states.

Actions: Left, Right, Suck.

Transition model:
All the Result(s, a)=s' possibilities

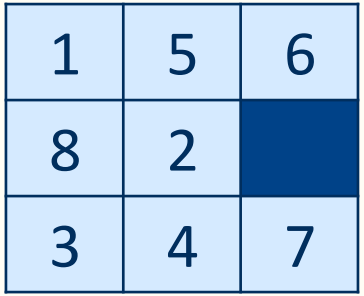
Goal test: Checks squares are clean.

Path cost: Each step costs 1.

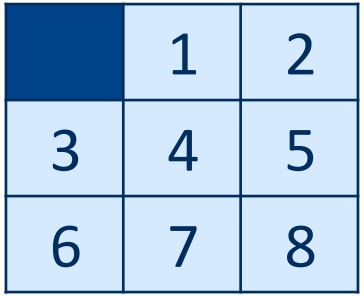


Left and Right have no effect in (respectively) the leftmost and rightmost positions.

Example: 8-piece puzzle



Initial state



Goal

What would the Problem Formulation look like?

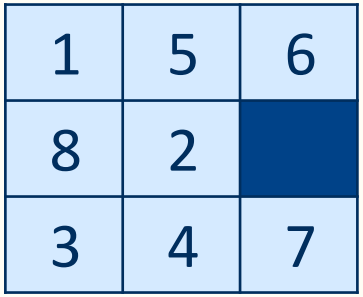
Example: 8-piece puzzle

States: Each state specifies the location of each tile and the blank. Any of these can be the initial state.

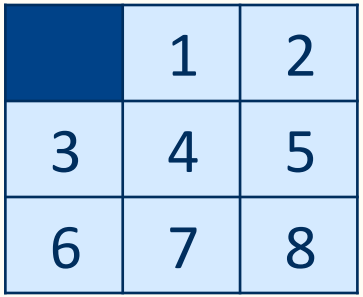
Actions: Simplest way to specify actions is to say what happens to the blank — Left, Right, Up and Down. Not all of these will be applicable in all locations of the blank.

Transition model: Gives the resulting state of each action. For example, Left in the initial state above switches the 2 and the blank.

Goal test: Checks if the goal configuration has been reached.
Path cost: Each step costs 1.



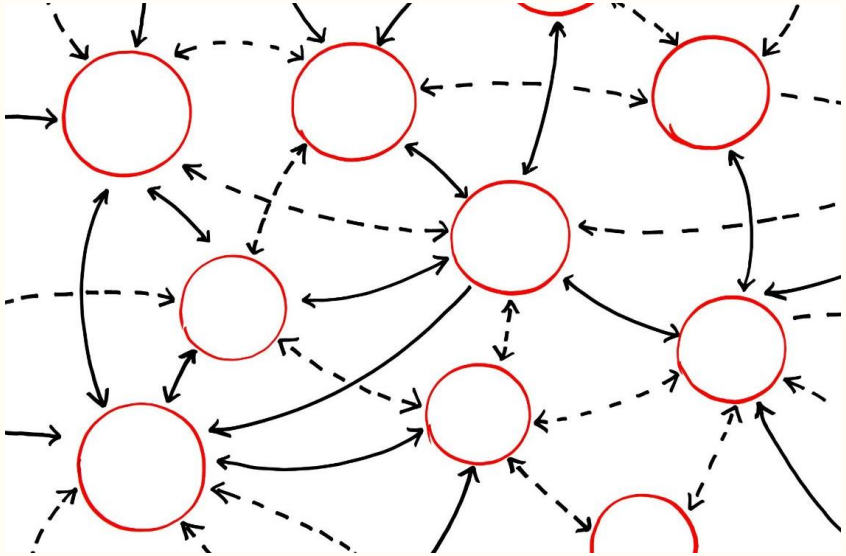
Initial state



Goal

State space

- State space is the set of all states reachable from the initial state.
- Can be pictured as a directed graph.
  - States are nodes
  - Actions are links
- Search the state space for a solution

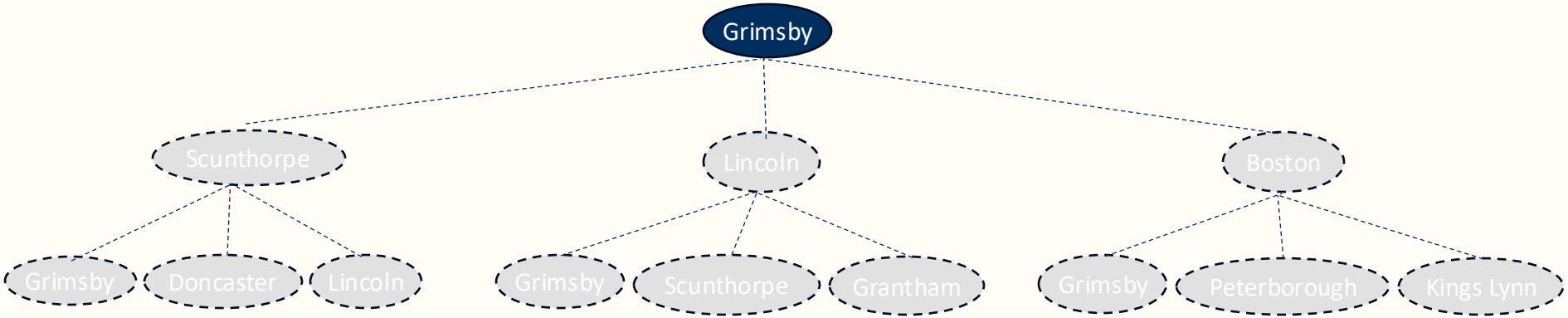


Searching from Grimsby to Nottingham using tree search...

(copilot, 2025)

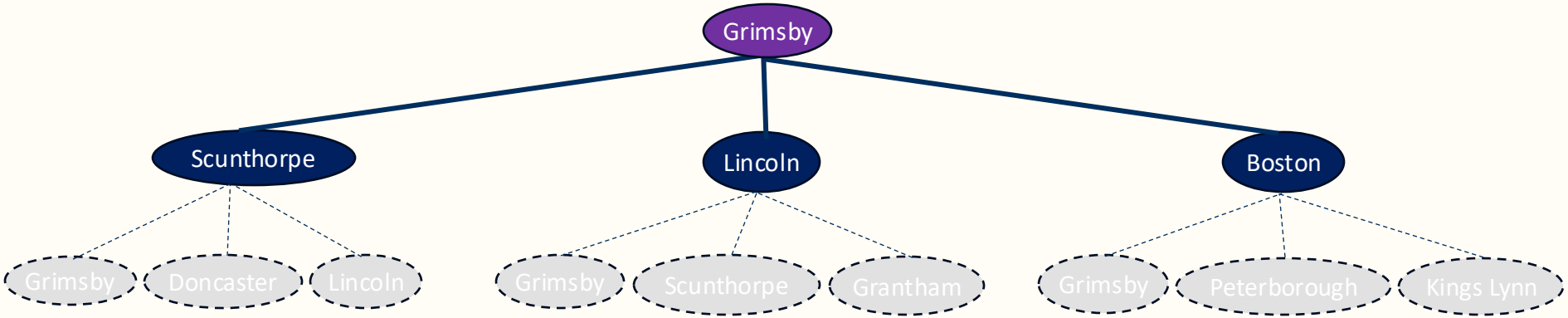
Tree search

1) Take the initial state and identify states that can be obtained by applying a single operator.



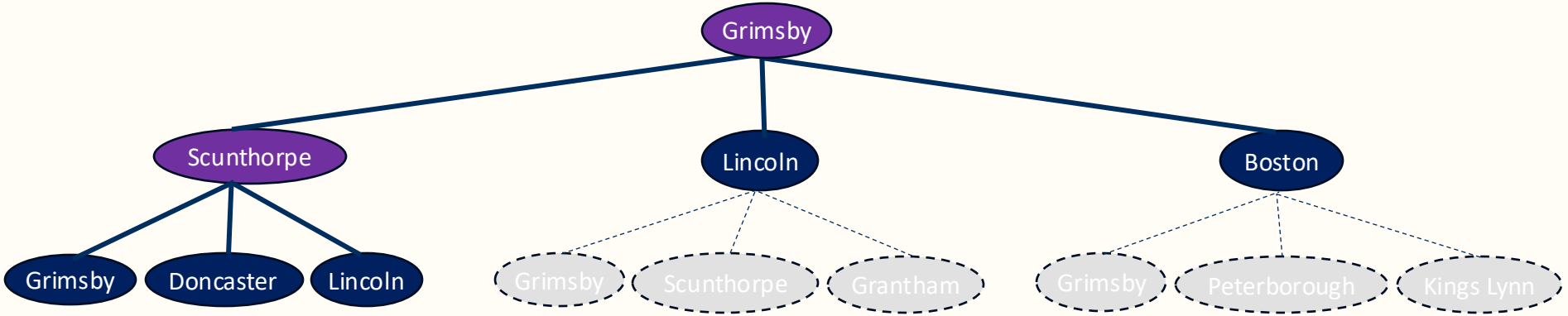
Tree search

2) These new states are the children of the initial state.
3) Test: are any of the children the goal?



Tree search

4) identify the children of the children



If we performed a tree search on a graph, we could end up revisiting a state multiple times.
Graph search still builds a tree but keeps track of explored states.

Search space vs search tree

State space is every possible state and the relationships between them.

- It is static.

Search tree the set of states the agent has looked at (is looking at) and some of the relationships between them.

- It is dynamic

Graph search

```
1:  function graph-search(problem) returns a solution or failure
2:  frontier ← [ Node(initial state) ]
3:  explored ← empty set
4:  loop do
5:     if  frontier is empty then return failure
6:     choose a node and remove it from frontier
7:     if node contains the goal then return plan
8:     add node to explored
9:     children ← expand node
10:    for each child in children
11:       if child not in frontier and not in explored
12:          add child to frontier
```

Search strategies

How do we pick states for expansion?

Many possibilities:

- Breadth-first
- Depth-first
- Iterative deepening
- Greedy
- A*
- D*, D*-Lite...

Still active area of research

Breadth first search

```
1:  function breadth-first-search(problem)
2:  node ← Node(problem.initialState)
3:  if problem.goalTest(node) then return solution(node)
4:  frontier ← FIFO (First In, First Out) queue with node as the only element
5:  explored ← empty set
6:  loop do
7:     if  frontier is empty then return failure
8:     node ← pop(frontier) /*pick shallowest node*/
9:     add node to explored
10:    for each  action in problem.actions(node)
11:       child ← childNode(problem, node, action)
12:       if child not in explored or frontier
13:          if problem.goalTest(child) then return solution(child)
14:          frontier ← AddToBack(child, frontier)
```



Breadth first search

BFS Exploration Method

BFS explores all nodes at the current depth before moving to the next -- uses a queue.

Completeness and Optimality

BFS is complete* and guarantees the shortest path if all actions have equal cost.

Memory Usage and Limitations

BFS requires significant memory to store all nodes at the current level, limiting its use for large state spaces.

Practical Applications

BFS is preferable when shortest path is essential and available memory is sufficient.

*is guaranteed to find a solution if one exists

(copilot, 2025)

Exponential

Breadth first search with branching factor = 10 and assuming 10 million nodes per second and each node needing 1000 bytes of storage.

Depth	Nodes	Time	Memory
2	110	0.011 msec	107 kilobytes
4	11,110	1.1 msecs	10.6 megabytes
6	10^6	0.11 secs	1 gigabytes
8	10^8	10 seconds	103 gigabytes
10	10^{10}	17 minutes	10 terabytes
12	10^{12}	27 hours	1 petabyte
14	10^{14}	4 months	99 petabytes
20	10^{20}	33,000 years	10 exabytes

Combinatorial explosion!

Depth first search

```
1:  function depth-first-search(problem)
2:  node ← Node(problem.initialState)
3:  if problem.goalTest(node) then return solution(node)
4:  frontier ← LIFO (Last In, First Out) stack with node as the only element
5:  explored ← empty set
6:  loop do
7:     if  frontier is empty then return failure
8:     node ← pop(frontier) /*pick deepest node*/
9:     add node to explored
10:    for each  action in problem.actions(node)
11:       child ← childNode(problem, node, action)
12:       if child not in explored or frontier
13:          if problem.goalTest(child) then return solution(child)
14:          frontier ← AddToFront(child, frontier)
```

Depth first Search

DFS Exploration Method

DFS explores each branch to its maximum depth before backtracking using a stack or recursion.

Completeness and Optimality

DFS is not guaranteed to find a solution if one exists. If it finds a solution, it may not be the optimal solution.

Memory Efficiency

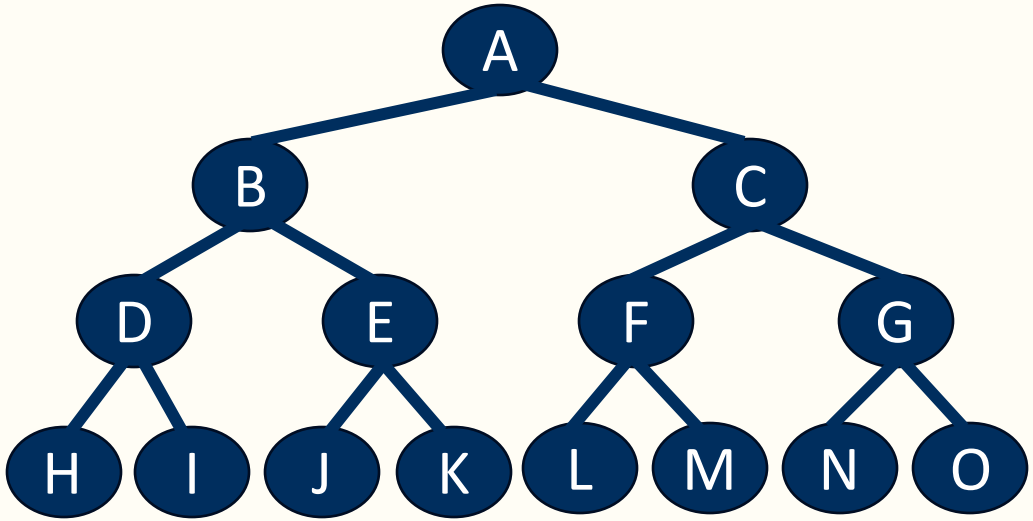
DFS requires memory only for the current path and unexplored sibling nodes, making it memory-efficient.

Practical Applications

DFS is useful in memory-limited scenarios and when solutions are deep within the search tree.

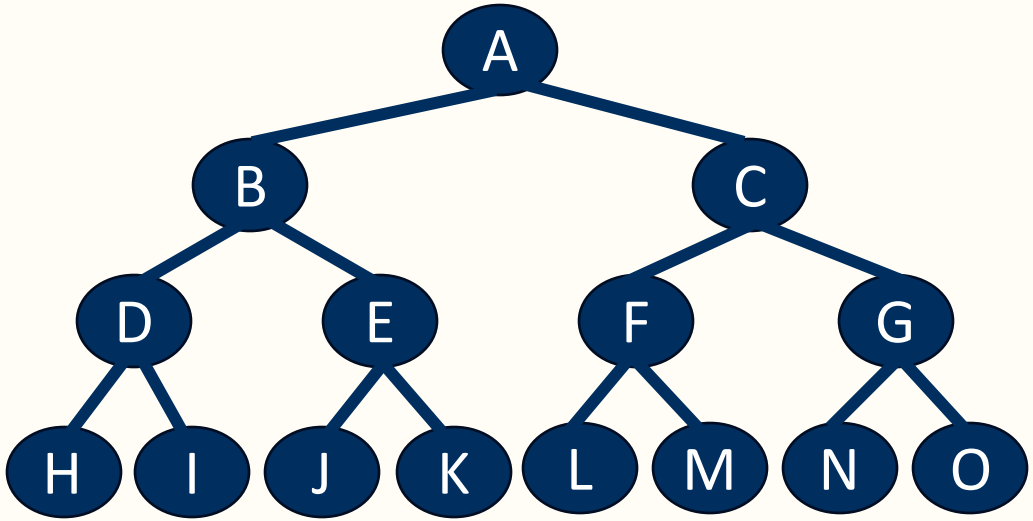
(copilot, 2025)

Which search methods is being used?



Nodes are explored in the following order: A, B, C, F, G, E, D, J, K, H, I, M, L, N, O

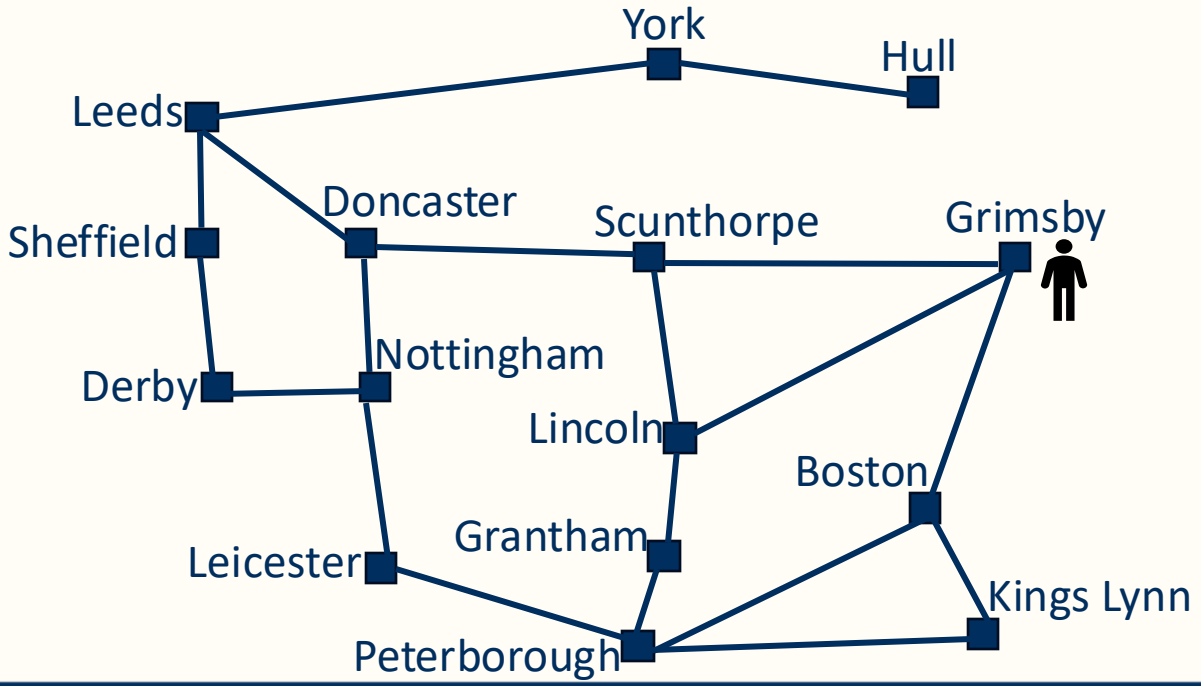
Which search methods is being used?



Nodes are explored in the following order: A, B, C, F, G, E, D, J, K, H, I, M, L, N, O

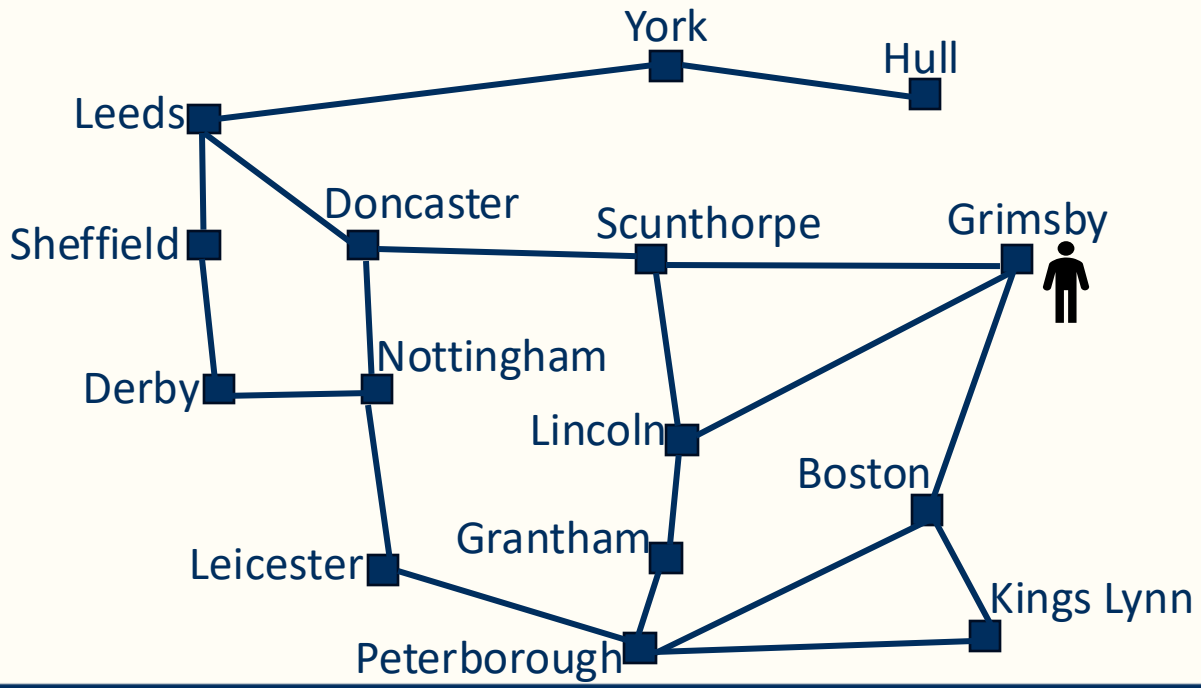
Answer: Breadth-first search

Which search methods is being used?



Plan returned is:
Go(Scunthorpe),
Go(Doncaster),
Go(Nottingham)

Which search methods is being used?



Plan returned is:
Go(Scunthorpe),
Go(Doncaster),
Go(Nottingham)

Answer: Could be either

Breadth and depth first search

Breadth-first search gives us optimality and completeness
But uses a large amount of memory.

Depth-first search uses less memory, but incomplete and has no guarantee of optimality.

Depending on the applications, we may know roughly how deep the solution is.

Depth-limited search

```
1:  function depth-limited-search(problem, l)
2:  node ← Node(problem.initialState)
3:  frontiers ← LIFO (Last In, First Out) stack with node as the only element
4:  explored ← empty set
5:  loop do
6:     if  frontiers is empty then return failure
7:     node ← pop(frontiers)
8:     add node to explored
9:     if problem.goalTest(node) then return solution(node)
10:    if node.depth > l
11:       cutoff
12:    else
13:       for each  action in problem.actions(node)
14:          child ← childNode(problem, node, action)
15:          if child not in explored or frontiers
16:             frontier ← AddToFront(child, frontier)
```

Depth-limited search

Depth Limit Concept

DLS modifies DFS by adding a depth limit. When the depth limit is reached, DLS returns a cutoff signal instead of continuing the search further.

Completeness and Optimality

Not complete as the solution may be deeper than the limit. Not optimal.

Memory Efficiency

DLS retains DFS memory efficiency.

Practical Use Cases

DLS is useful when solution depth is known or estimated.

(copilot, 2025)

Iterative deepening search

```
1: function iterative-deepening-search(problem)
2: for depth = 0 to ∞ do
3:     result ← depth-limited-search(problem, depth)
4:     if result ≠ cutoff then return result
```

Iterative deepening search

Depth Limit Concept

IDS combines the space efficiency of DFS with the completeness of BFS by repeatedly performing DFS with increasing depth limits.

Completeness and Optimality

IDS is complete and finds the shortest path to the goal if all actions have equal cost, like BFS.

Memory Efficiency

IDS uses memory proportional to the depth limit, making it highly memory-efficient for large search spaces.

Practical Use Cases

IDS is ideal when the solution depth is unknown and memory resources are limited, such as in puzzle solving or pathfinding in large graphs.

Might take more time than BFS but saves a lot of memory.

(copilot, 2025)

Iterative deepening search

Example: branching factor = 10 and solution depth = 5.

Breadth first search:

- examines 111, 111 nodes
- memory requirement of 100, 000 nodes.

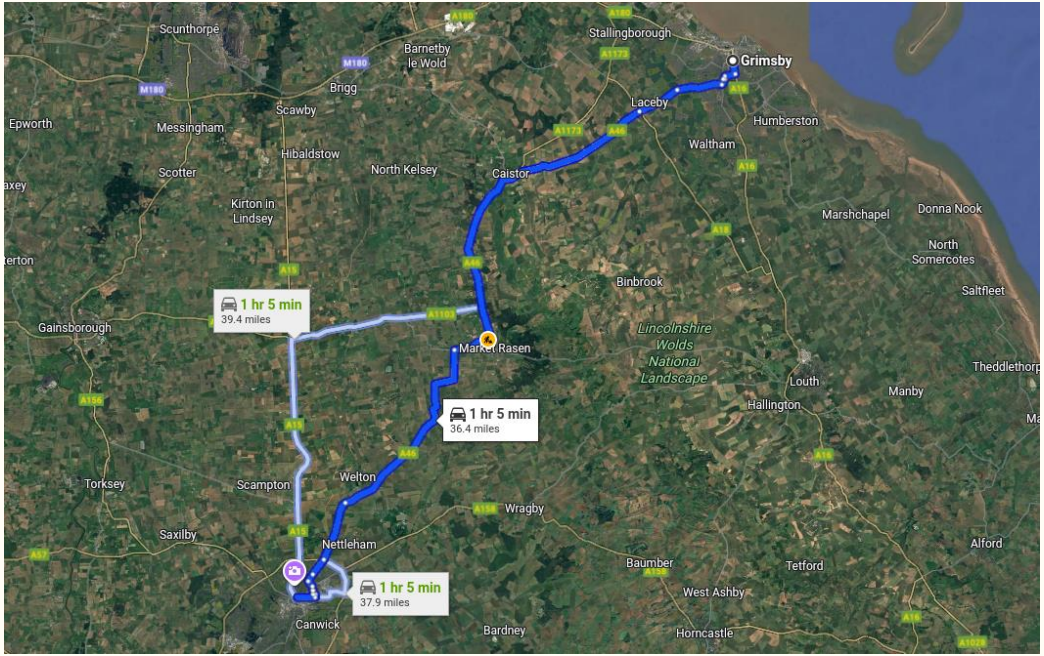
Iterative deepening:

- examines 123, 456 nodes
- memory requirement of 50 nodes.

Takes 11% longer in this case. Uses 2000 times less memory.

A help, but still not enough — doesn't address the 33,000 years on earlier slide.

Does not scale to GMaps



(Google, 2025)

Google (2025) Grimsby to Lincoln driving [Online] Available from: http://google.com/maps (Accessed: 4 Oct 2025)

Search method comparison

Method	Complete	Optimal
BFS	Yes	Yes
DFS	No	No
DLS	No	No
IDS	Yes	Yes

When all action have equal cost

(copilot, 2025)

Lecture Learning Outcomes

By the end of this lecture, you will be able to:

- Understand search as a problem-solving technique in AI.
- Represent problems as graphs.
- Explain DFS, BFS, DLS and IDS.
- Compare the strengths and weaknesses of each method.

Workshop: implement and compare the different search methods.

Extra reading: R&N Chapter 3 Solving Problems by Search (up to end of section 3.4)

Extra Activity: Trace through DFS, BFS, DLS and IDS using our Grimsby--> Nottingham example.

For help with the extra activity, drop by my office hours or talk to me in the workshop.

References

These slides were produced using material from:

- Microsoft Copilot (2025). "Imagine you are teaching second year undergraduate students. Write the slides for a lecture that covers: search as problem solving, graph search, depth-first search, breadth-first search and depth-limited search. Include information on the advantages and disadvantages of the search methods. Include interactive elements suitable for a large lecture theatre." "Create a similar slide to this but for Iterative deepening search" [Large Language Model]. Available at: https://copilot.microsoft.com/ (Accessed: October 4, 2025).
  - Note, most the output wasn't that useful.
  - The only content Copilot created that was used was: the decorative picture and format of slide 6, the structure of slide 7, the decorative picture on slide 22, the structure of slides 30, 33, 40 & 42 and the format of the table on slide 45.
- Parsons, S. (2024) Search [Lecture] Artificial Intelligence CMP2020-2425 University of Lincoln, Oct 2024.
- Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, Global Edition. Pearson.